InnovShop

Un site e-commerce de produits innovants, de la conception à la mise en ligne

Projet final · Développeur Web et Web Mobile

Nicolas Teixeira · Soutenance du 2 novembre 2026

LE PROJET

Un site e-commerce complet, testé et en ligne

La boutique

Catalogue, panier, commande en 4 étapes avec paiement, espace client et administration de la boutique.

Ce qui la distingue

Une marketplace pour des vendeurs partenaires, et une catégorie « Prototypes » réservée aux testeurs.

Livré

Site hébergé, dépôt GitHub avec README, backlog Trello, dossier de projet et 60 tests automatisés.

Cinq profils d'utilisateurs : visiteur, client, vendeur, testeur, administrateur.

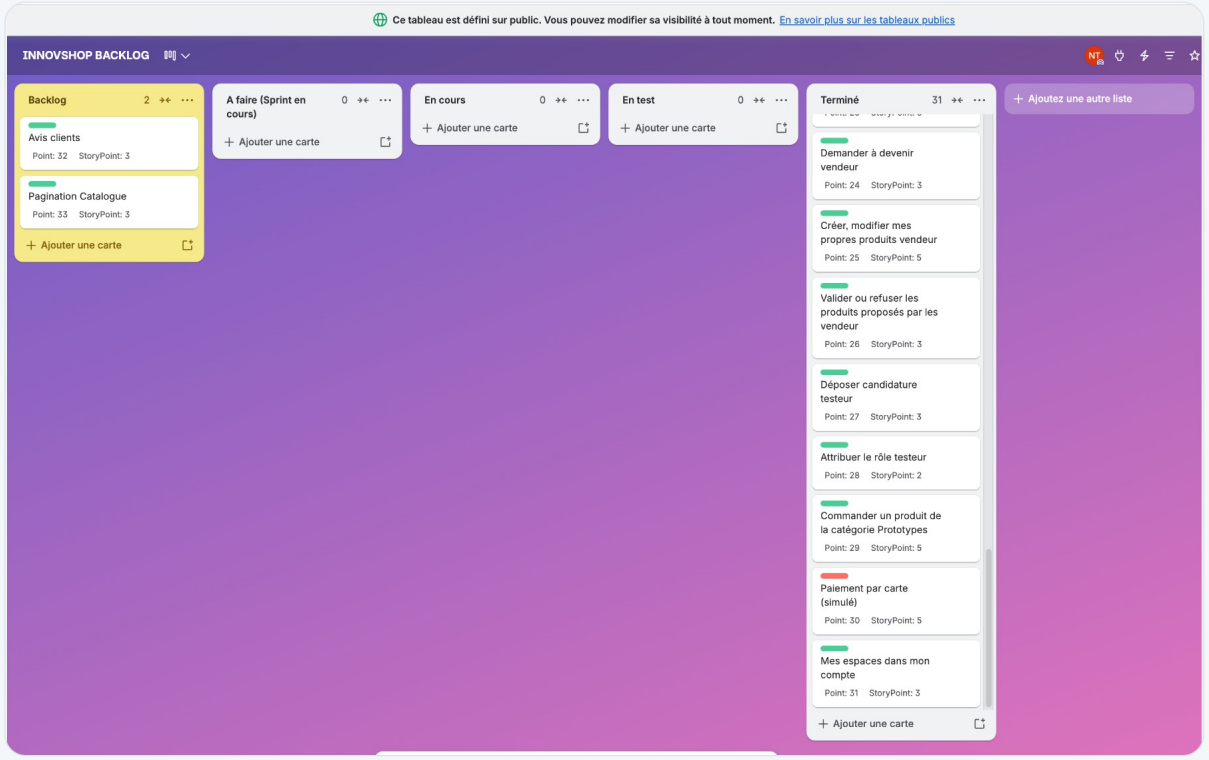
MÉTHODE

Un backlog de 31 user stories suivi sur Trello

31

user stories : catalogue, panier, commande, administration, marketplace, prototypes

Priorités en MoSCoW, estimation en story points, tableau Kanban public.



TECHNOLOGIES

Symfony et ses outils, en architecture MVC

Serveur

Symfony 7.4, PHP 8

Données

MariaDB, Doctrine ORM, migrations

Interface

Twig, Bootstrap 5, Turbo, AssetMapper

Administration

EasyAdmin 5

Qualité

PHPUnit, PHP-CS-Fixer, Git et GitHub

Hébergement

Alwaysdata, e-mails par SMTP

Le code suit le chemin : contrôleur, service, repository, entité.

ENVIRONNEMENT DE DÉVELOPPEMENT

Les outils pour développer, tester et publier

Sur mon poste

- Visual Studio Code
- PHP 8.2 et Composer
- Symfony CLI : serveur local
- XAMPP : base MariaDB

Qualité et suivi

- Git et GitHub : versions
- Trello : backlog Kanban
- PHPUnit : tests
- PHP-CS-Fixer : style
- diagrams.net : diagrammes

En ligne

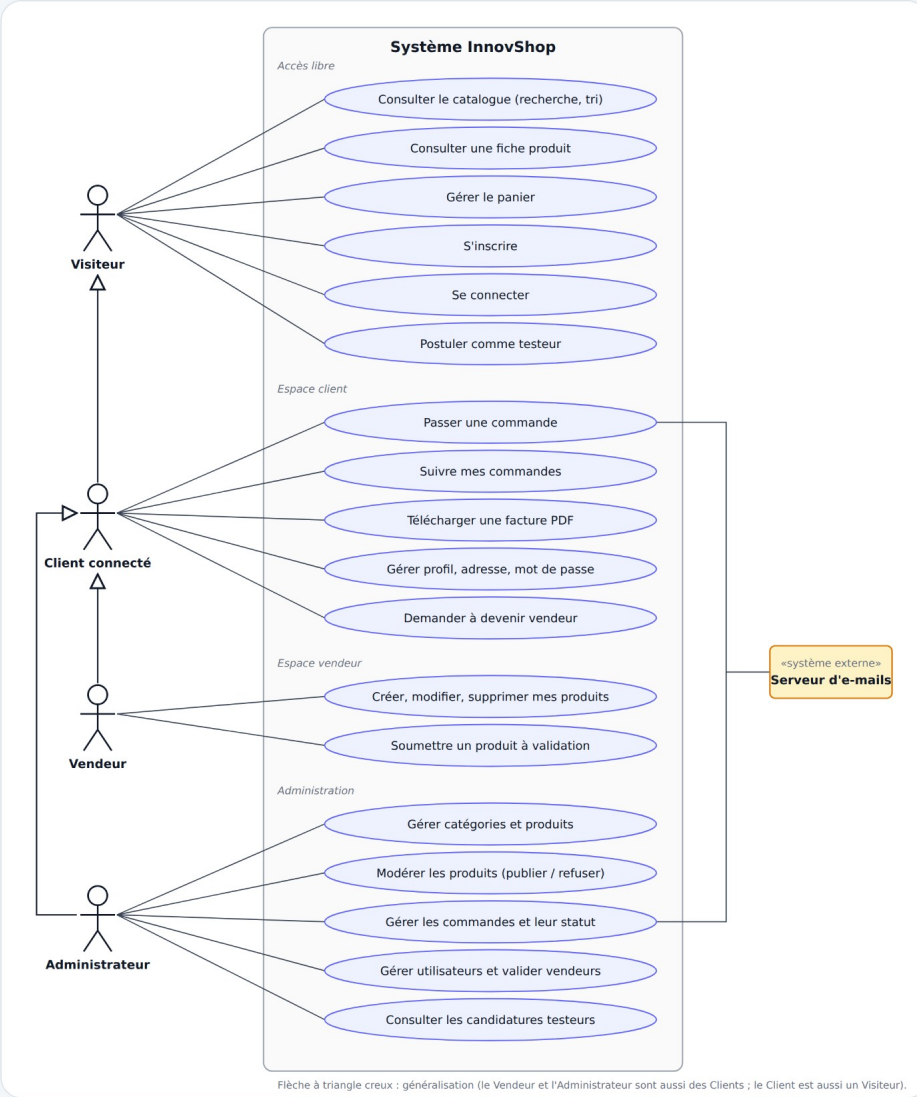
- Alwaysdata : hébergement mutualisé
- Accès SSH pour les mises à jour
- Fichier .env.local : secrets uniquement sur la machine

Je vous montre le projet ouvert dans Visual Studio Code.

CONCEPTION · 1

Qui peut faire quoi : les cas d'utilisation

- **Visiteur** : parcourt le catalogue, remplit un panier, s'inscrit
- **Client** : commande, suit ses commandes, télécharge sa facture
- **Vendeur** : gère ses propres produits
- **Testeur** : commande les prototypes (rôle donné après candidature)
- **Administrateur** : gère la boutique et modère les produits

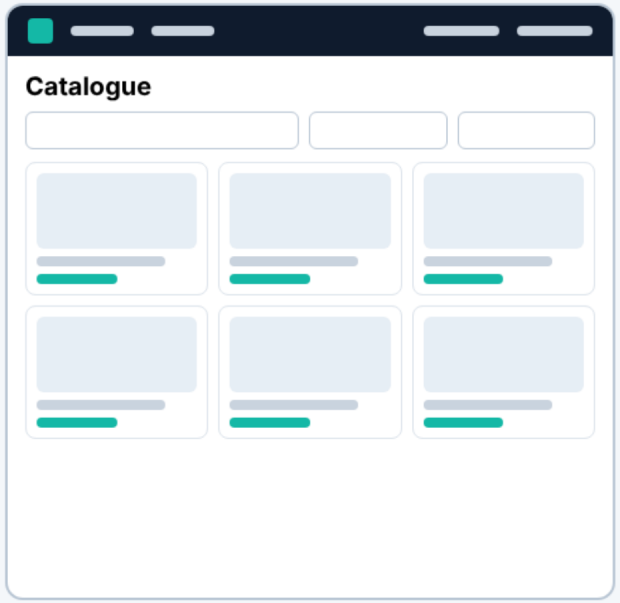


INTERFACE · MAQUETTES

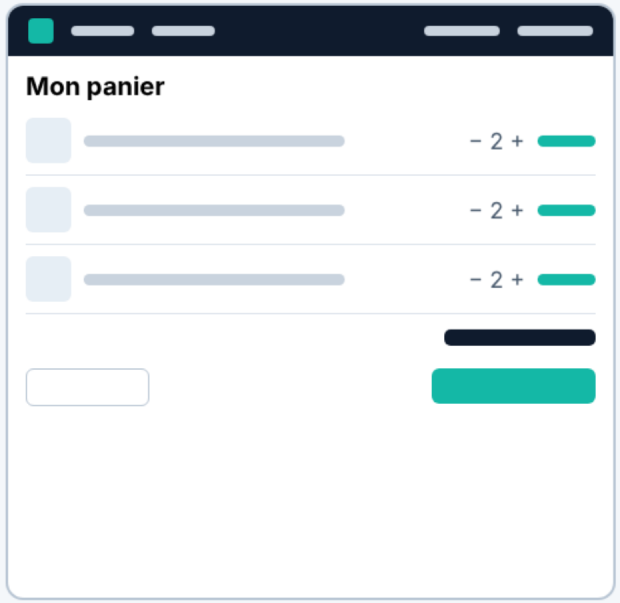
Les maquettes des écrans principaux



Accueil (ordinateur)



Catalogue : recherche, filtre, tri



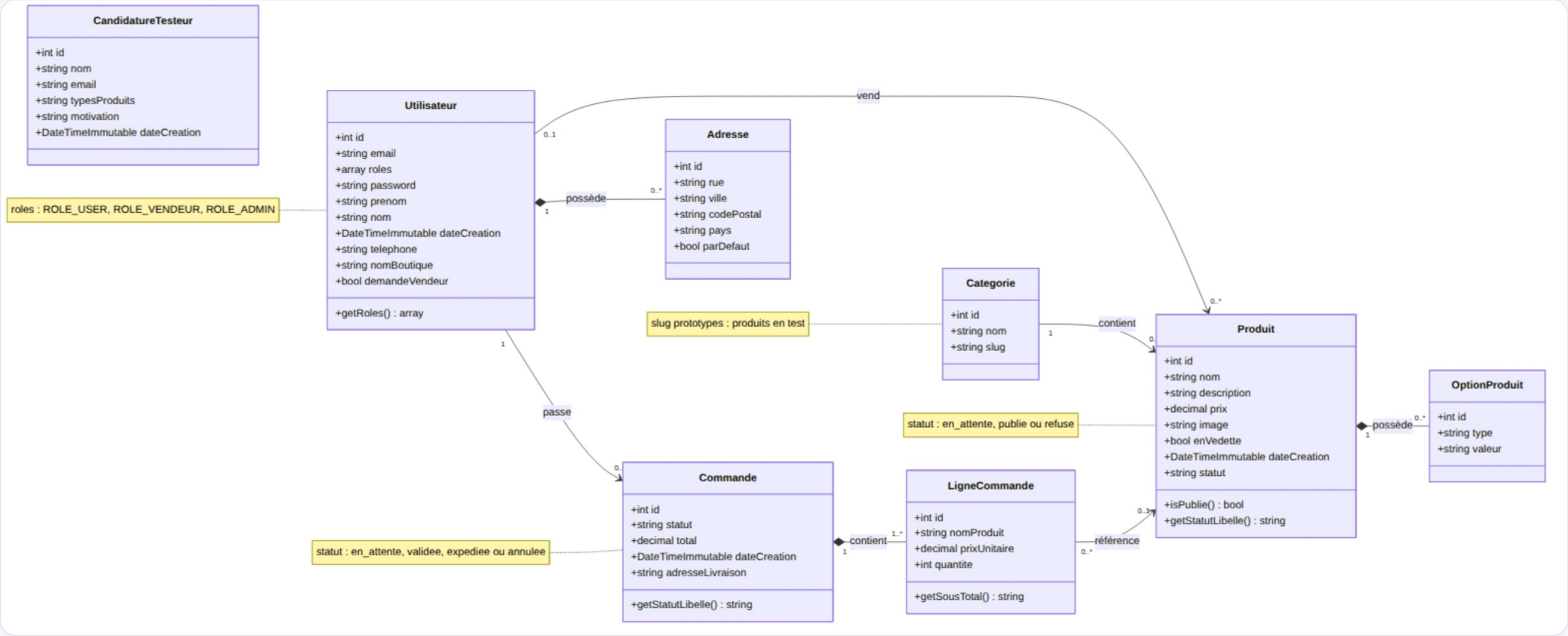
Panier



Accueil (mobile)

Même trame partout : barre de navigation sombre, contenu centré, boutons d'action en turquoise et orange.

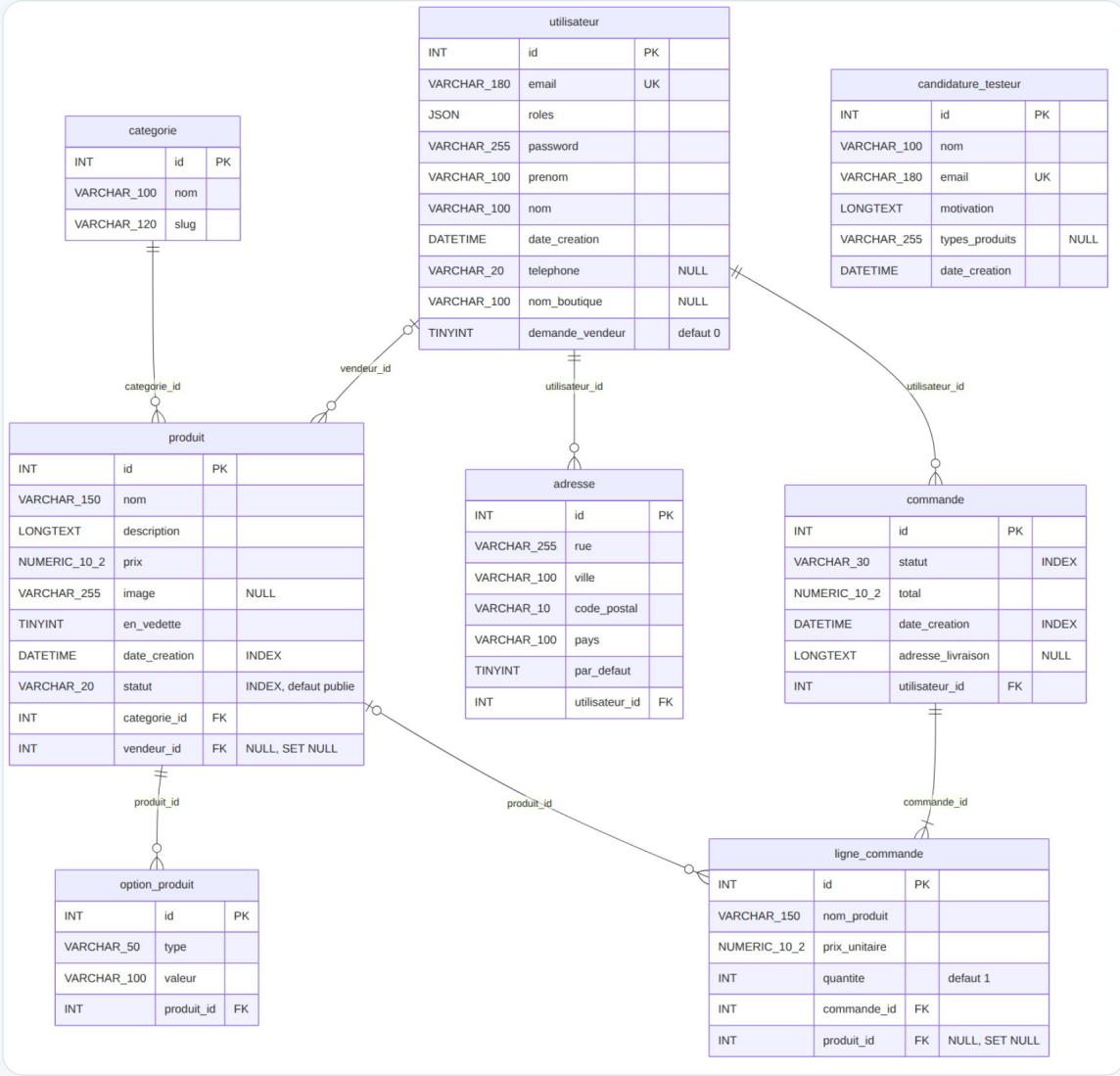
Le diagramme de classes



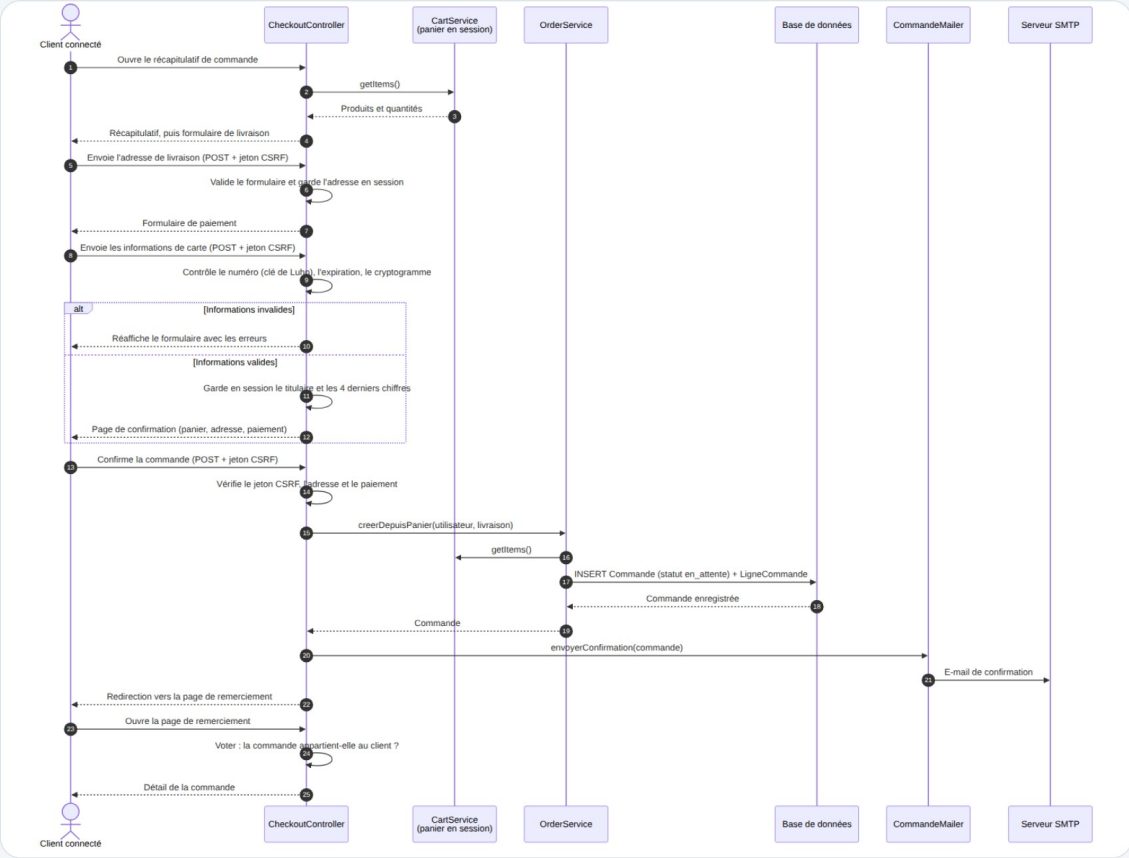
CONCEPTION · 3

Le modèle physique de données

- 8 tables métier, reliées par des clés étrangères
- La ligne de commande garde le nom et le prix du produit au moment de l'achat
- Des index sur le statut et la date de création
- Généré par migrations Doctrine



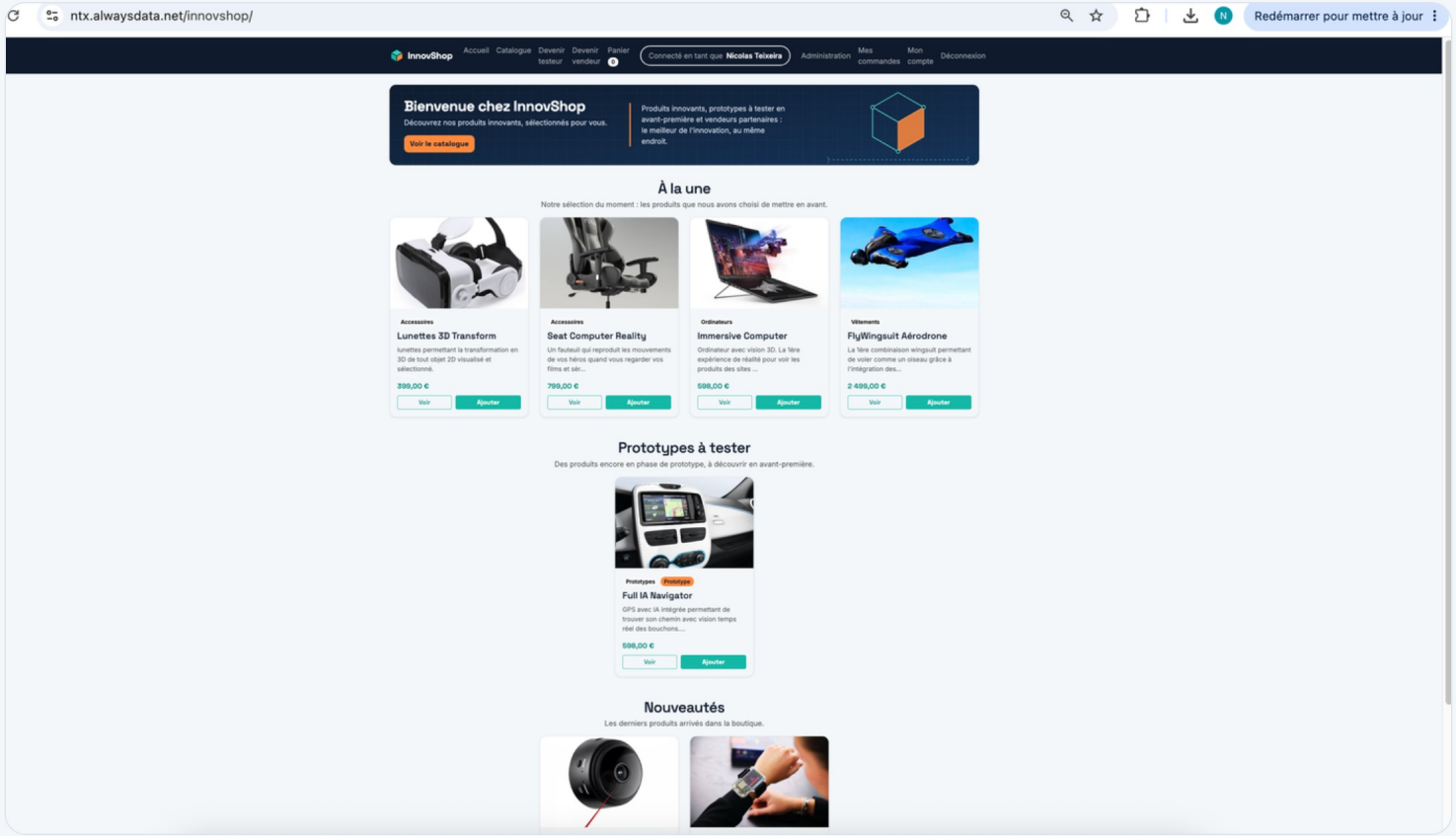
Le scénario clé : passer une commande



- 4 étapes : panier, livraison, paiement, confirmation
- Carte contrôlée, aucune donnée de carte conservée
- La commande n'est créée qu'à la confirmation
- Un e-mail récapitulatif est envoyé

RÉALISATION · 1

Une identité visuelle pensée pour le projet



Encre, turquoise, orange : une palette de trois couleurs.

Space Grotesk et Inter

Logo, bandeau, cartes produit avec survol : tout est dans une feuille de style unique.

Un contrôleur : le panier

```php
#[Route('/panier')]
class CartController extends AbstractController
{
    #[Route('/ajouter/{id}', name: 'app_cart_add', methods: ['POST'])]
    public function add(
        Produit $produit,
        Request $request,
        CartService $cart,
    ): Response {
        $this->verifierJeton($request);
        $this->denyAccessUnlessGranted(ProduitVoter::COMMANDER, $produit);
        $cart->add($produit->getId());
        return $this->repondre($request, $cart, 'Ajouté au panier.');
    }
}
```

Extrait abrégé de src/Controller/CartController.php

1. **Route** : l'adresse /panier/ajouter/12, en POST seulement
2. **Paramètres** fournis par Symfony : le produit lu en base, la requête, le service
3. **Jeton CSRF** contrôlé avant toute action
4. **Voter** : le droit de commander ce produit
5. **Service** : la logique du panier reste hors du contrôleur

Twig : séparer l'affichage de la logique

templates/base.html.twig (gabarit commun)

```
<a href="{{ path('app_cart') }}">
    Panier <span id="cart-count">
        {{ cart_count() }}
    </span>
</a>
{% if is_granted('ROLE_ADMIN') %}
    <a href="{{ path('admin') }}">
        Administration
    </a>
{% endif %}
```

templates/cart/index.html.twig

```
{% extends 'base.html.twig' %}

{% block body %}
    {% for item in items %}
        <td>{{ item.produit.nom }}</td>
        <td>
            {{ item.produit.prix|number_format(2, ',', ' ') }} €
        </td>
    {% endfor %}
{% endblock %}
```

- **{{ ... }}** affiche une valeur, **{% ... %}** exécute une instruction (boucle, condition, héritage)
- **extends** et **block** : chaque page hérite du gabarit commun
- Les valeurs affichées sont échappées automatiquement : protection contre l'injection de scripts

RÉALISATION · JAVASCRIPT

JavaScript : un panier sans rechargement

```
document.addEventListener('submit', async (event) => {
    const form = event.target.closest('form[data-cart-form]');
    if (!form) { return; }
    event.preventDefault();
    const reponse = await fetch(form.action, {
        method: 'POST',
        body: new FormData(form),
    });
    const data = await reponse.json();
    majCompteur(data.count);
    notifier(data.message);
    await rafraichirPanier();
});
```

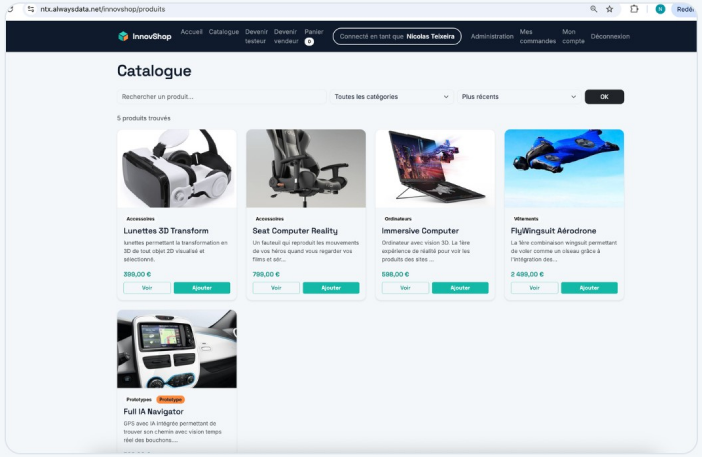
Extrait abrégé de assets/cart.js · sans JavaScript, le formulaire fonctionne quand même

1. On écoute l'envoi des formulaires du panier
2. **preventDefault** : la page ne se recharge pas
3. **fetch** envoie le formulaire, jeton CSRF compris
4. Le serveur répond en JSON : compteur et message
5. Le compteur, la notification et le tableau sont mis à jour

RÉALISATION · 2

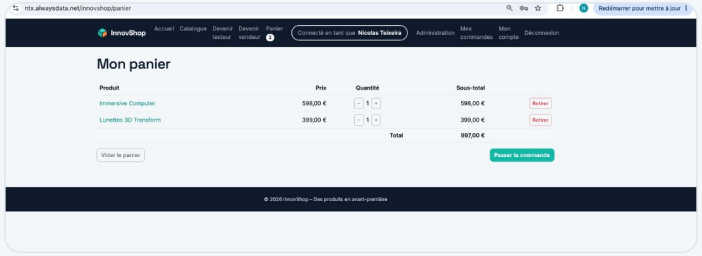
Du catalogue à la commande

1 · Catalogue



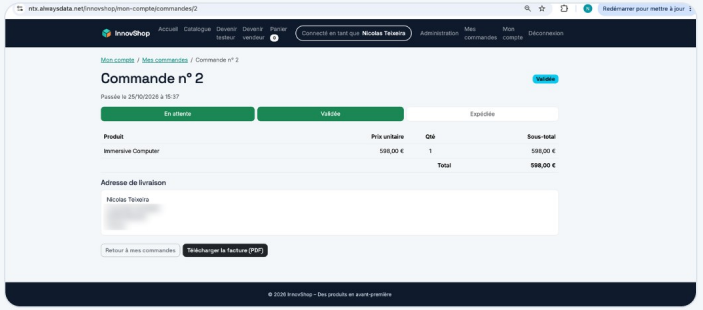
Recherche, filtre par catégorie, tri

2 · Panier



Quantités modifiables, total recalculé

3 · Commande



Suivi du statut et facture PDF

RÉALISATION · 3

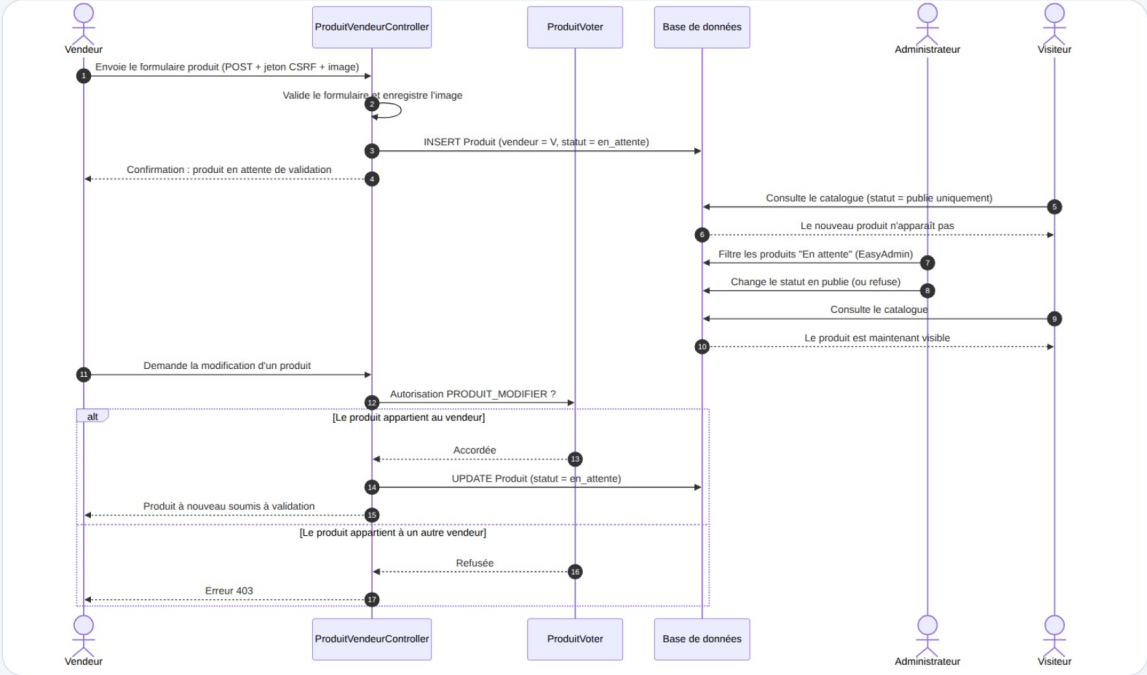
Marketplace et prototypes

Marketplace

Un vendeur propose un produit, qui passe « en attente ». L'administrateur le publie ou le refuse.

Prototypes

Un visiteur candidate pour devenir testeur. Seuls un testeur validé ou un administrateur peuvent commander un prototype.



SÉCURITÉ

Chaque risque a sa parade

Risque	Mesure
Accès non autorisé	Rôles hiérarchisés, règles d'accès par zone, Voters
Faux envoi de formulaire (CSRF)	Jeton CSRF sur les formulaires et actions sensibles
Mots de passe	Hachés, règle de robustesse à l'inscription
Injection et scripts malveillants	Requêtes Doctrine, texte nettoyé, caractères < et > refusés
Comptes en double	E-mail unique, contrôlé par validation
Prototypes pour tous	Voter réservé aux testeurs et administrateurs

TESTS ET QUALITÉ

Des tests automatiques qui protègent le site

60
tests

125
assertions

0
échec

- Catalogue, panier, commande, droits d'accès, CSRF, marketplace, prototypes
- Base de test dédiée, chaque test annulé à la fin
- Style du code vérifié avec PHP-CS-Fixer

DÉPLOIEMENT ET PERFORMANCE

Une mise à jour en cinq étapes sur Alwaysdata

1 Récupérer le code (git pull)

2 Installer les dépendances (si besoin)

3 Appliquer les migrations (s'il y en a)

4 Compiler les ressources

5 Vider le cache

- Configuration de production uniquement sur le serveur, jamais dans GitHub
- Fichiers statiques en cache un an et compressés (gzip)
- Base de données en production : MariaDB 11.4
- Audit Lighthouse en mobile : performance 85, accessibilité 92, bonnes pratiques 100, SEO 83

BILAN

Ce que j’ai appris, ce qui reste à faire

Difficultés

- Quota disque plein en production : compilation bloquée, caches supprimés
- Une migration oubliée : table manquante en production
- Ressources non recompilées : site sans style
- Droits d'accès à garder cohérents partout

Enseignements

- Toujours vérifier en production, pas seulement en local
- Les tests évitent les régressions
- Documenter aide à reprendre le projet

Évolutions

- Paiement en ligne réel
- Avis clients
- Pagination du catalogue
- Points relevés par l'audit Lighthouse

DÉMONSTRATION

Voyons le site en ligne

1. Un client ajoute un produit au panier et commande
2. Un vendeur propose un produit, l'administrateur le publie
3. Un prototype : réservé aux testeurs

ntx.alwaysdata.net/innovshop

Merci de votre attention. Des questions ?